

Post-Quantum TLS Under Pressure: Measuring Handshake DDoS Risk for ML-KEM / ML-DSA Deployments

Bryce Vitale

Cybersecurity M.S. Capstone Thesis

Abstract—Post-quantum cryptography (PQC) is being standardized for real deployment, which makes TLS migration a near term engineering problem rather than a purely theoretical one. The TLS handshake naturally has an availability bottleneck because it concentrates asymmetric operations and certificate processing during connection establishment, making it a target for handshake layer flooding. This paper evaluates whether moving toward PQC ready TLS materially shifts handshake overhead in ways that change susceptibility to handshake DDoS, and what mitigation choices can reduce risk without undermining migration goals. We built a reproducible Ubuntu testbed using OpenSSL 3.0.2 with the Open Quantum Safe (OQS) provider loaded via the OpenSSL provider interface. Using controlled OpenSSL `s_server/s_client` handshakes, we validated negotiation of the hybrid key agreement group X25519MLKEM768 and quantified handshake bytes and handshake timing relative to X25519. Separately, we tested a TLS endpoint under high rate connection churn and showed that practical system limits like backlog behavior materially shape observed failure thresholds under handshake heavy conditions. In this testbed, the hybrid group increases handshake bytes in both directions and increases mean handshake time ($N = 160$), supporting the broader conclusion that PQC ready TLS is not only a cryptographic choice but also an availability engineering problem that requires handshake stress testing and configuration hardening as part of deployment readiness.

Keywords— *TLS 1.3, post-quantum cryptography, ML-KEM, ML-DSA, hybrid key exchange, handshake flood, denial of service.*

I. INTRODUCTION

Large scale quantum computing threatens classical public key cryptography because Shor's algorithm breaks the hardness assumptions behind RSA and elliptic curve discrete logarithms, which are foundational to today's Internet authentication and key agreement. [8] This threat changes how confidentiality should be framed. Once a capable quantum computer exists, encrypted traffic may be decrypted later. This means that migration is driven by durability rather than present time compromise. NIST's standardization of

ML-KEM (FIPS 203) and ML-DSA (FIPS 204) makes PQC adoption a near term engineering reality, not a speculative research topic. [2], [3]

This analysis focuses on the availability side of PQC adoption. If PQC increases handshake overhead, then the handshake layer that is flooding the failure threshold may shift and mitigation requirements may change. [11] Below is a short roadmap for the structure of the paper.

Section II reviews TLS 1.3 handshake mechanics, NIST PQC standardization, and hybrid key exchange. Section III defines the problem statement, threat model, and evaluation metrics used to quantify handshake flood risk. Section IV describes the testbed and methodology, including provider enablement and measurement procedures. Section V presents measured handshake overhead and handshake flood results. Section VI discusses implications for PQC-ready deployments, and Section VII outlines related work. Section VIII talks about limitations and future work. Section IX concludes.

This project is intentionally framed as a deployment risk measurement problem. PQC is not useful to an operator if it quietly turns the handshake into the easiest place to knock a service offline. In practice, handshake layer attacks do not need to “break cryptography”. They win by forcing worst case behavior over and over. Pushing new connections and new handshakes along with repeated certificate processing until the server runs out of something it cannot scale instantly. This includes resources like CPU utilization and accept queue capacity. [7]

TLS 1.3 is designed to reduce handshake complexity. However, it must first parse the *ClientHello* to then negotiate a group. The next step would be to allocate a connection state and complete the authenticated handshake before any application level control can help. [1] That is why this paper treats the handshake itself as the system boundary. If the handshake cost profile changes, the availability risk also changes.

PQC adds pressure because it increases what gets exchanged and processed during the handshake. Even if ML-KEM is computationally efficient, hybrid

negotiation increases the amount of key material and transcript processing relative to a classical ECDHE-only handshake. [4] Existing measurement work already shows post-quantum TLS can create significant message size differences and client/server asymmetries, which are exactly the conditions attackers look for when they want to amplify impact per request.

A useful way to check handshake risk is to compare with newer transport designs. QUIC includes an explicit anti amplification rule before address validation because early handshake responses can otherwise be exploited as an amplification vector. [12] TLS over TCP does not enforce QUIC’s transport rule, so the burden is on deployment configuration and mitigation strategy. This work measures what is concrete and observable. Bytes exchanged during the handshake and time per handshake will determine failure behavior under a handshake heavy load. When a failure happens, we treat it as if a system asks a question like, “what resource capped?” and “did PQC overhead shift the cap?”. To keep the results in scope, we do not include beyond what is measured in the testbed. Where we discuss broader implications, we explicitly tie them back to standards and prior measurement work rather than speculation. The remainder of the paper follows a simple flow, explaining establishing the protocol and deployment background and define a threat model. From there comparison metrics are used where we can measure classical vs hybrid handshake overhead to interpret what those measurements imply for handshake flood resilience.

Contributions.

1. We built a TLS 1.3 testbed that enables hybrid PQ key exchange using OpenSSL 3’s provider interface and oqsprovider. [6]
2. We measured handshake byte and timing overhead for X25519 versus the hybrid group X25519MLKEM768 in this testbed and reported the results with traceable artifacts.
3. We evaluated handshake flood behavior and showed that practical host limits (file descriptors and connection concurrency settings) can dominate failure thresholds under churn, which belongs in PQ deployment hardening. [7]

II. BACKGROUND

This section summarizes the protocol mechanics and how the literature directly motivates this project. A few topics that will be covered in this section are, how TLS 1.3 handshakes work, what NIST standardized, what hybrid migration means in practice, and what prior work reports about PQ-TLS overhead.

A. TLS 1.3 Handshake

TLS 1.3 establishes a shared secret during the handshake and then derives symmetric traffic keys. This is when the ClientHello/ServerHello exchange negotiates the key share group and the server authenticates with a certificate chain and transcript signature (CertificateVerify). [1] Because the handshake includes asymmetric operations and state allocation, it tends to be the most expensive part of a connection under churn, which is why it is also a common target in availability attacks. [7]

B. ML-KEM & ML-DSA

ML-KEM is a module-lattice-based key encapsulation mechanism standardized by NIST (FIPS 203) for establishing shared secrets over an untrusted channel. [2]

ML-DSA is a module-lattice-based digital signature scheme standardized by NIST (FIPS 204) intended for post-quantum authentication, including certificate ecosystems. [3]

In TLS deployments, ML-KEM primarily affects the key agreement path (key share size and processing), while ML-DSA affects authentication (certificate sizes and signature verification). [10]

C. Hybrid Key Exchange

Hybrid key exchange combines classical ECDHE with a PQ KEM so that the final secret remains secure if either component remains unbroken. This is one of the most practical transition strategies during early PQC adoption. Hybrid approaches are widely discussed because they reduce migration risk while maintaining forward momentum toward quantum resistance, but they also increase handshake bytes and computational work compared to a purely classical handshake. [5]

Sosnowski et al. provide a comprehensive performance analysis of post-quantum TLS 1.3 and highlight that PQ and hybrid handshakes can introduce asymmetry and message size amplification that may matter under attack scenarios. [5]

D. Deployment Pressure & Operation

NIST standardization is the turning point that forces operational teams to treat PQC as a timeline problem. Once standardized algorithms exist, the remaining barrier is integration cost and operational risk, not academic uncertainty about whether the primitive is “real.” Mosca’s readiness framing is useful in framing the idea that migration is not a single flip, instead, it is a staged process with inventory, prioritization, and rollout phases that can take years in real enterprises. [9]

TLS is a top priority in that process because it protects the largest volume of real traffic and it is the layer most exposed to external adversaries. From an engineering standpoint, this means two things. First,

PQC should be validated in real stacks (OpenSSL/Nginx and deployment like pipelines), via microbenchmarks. Second, the availability impact must be evaluated under adversarial conditions, not just under steady state load, because handshake layer attacks specifically target churn and worst case behavior. Sikeridis et al. evaluate post-quantum authentication in TLS 1.3 and show that PQ certificate chains and signature schemes can increase handshake overhead and reduce throughput depending on network conditions and configuration. [10] Rios et al. benchmark post-quantum TLS in a modern web context and reinforce that deployment decisions must consider latency/throughput tradeoffs rather than only algorithmic security arguments. [11] Kampanakis and Childs-Klein examine post-quantum TLS overhead from a “time to last byte” perspective and show that ‘handshake cost’ interacts with link conditions and transferred data volume. [15]

Prior work is also useful because it separates where overhead comes from. In practice, PQ-TLS cost is a combination of (i) larger key shares, (ii) certificate chain payload growth, and (iii) signature and verification costs, all of which land inside a single handshake transcript. [10] These matter for availability because attackers do not need to predict which component is expensive, they just need the handshake to be expensive enough that repeated handshakes become the fastest way to burn server resources.

A key gathering from the CoNEXT measurements is that overhead is not uniform across algorithm choices. Some PQ signature choices produce much larger handshake messages than others, and the server-side workload can be significantly higher than the client-side workload, which is the exact asymmetry that can become dangerous under flood traffic. [5] Furthermore, for authentication, the migration path is not only “swap the signature algorithm.” In X.509 ecosystems, certificate chain size and intermediate structure can change the bytes exchanged and the verification path. Mixed chain transition strategies have been proposed specifically to reduce ecosystem breakage while still introducing PQ authentication, but those strategies still need availability testing because they change handshake payload structure. [16]

Additionally, embedded and constrained environments are important because they represent a worst case operational baseline. If a server class CPU is already close to a threshold under handshake churn, that threshold will be lower on constrained devices, and PQ overhead will show up more aggressively. [17] Hybrid PQ TLS has been evaluated on embedded devices as well, reinforcing that the cost profile is sensitive to implementation and environment, not just the abstract algorithm. [18] The IEEE Network benchmarking work is useful here because it frames PQ-TLS overhead as a

deployment and tuning question, not a one off microbenchmark, which is consistent with how operators actually decide whether a configuration is safe to roll out.

E. Handshake Flooding & Mitigation

When it comes to Handshake Flooding, denial of service is fundamentally about pushing systems into worst-case resource consumption. This is when attackers exploit algorithmic or stateful bottlenecks rather than average case behavior. Modern transports recognize handshake layer abuse explicitly and QUIC limits server response amplification prior to address validation to reduce spoofed source handshake amplification risk. [12]

QUIC uses TLS 1.3 for its cryptographic handshake, but its transport design changes the shape of handshake exposure relative to TLS over TCP, which is useful as a reference point for availability analysis. [13]

DTLS 1.3 includes a cookie exchange to delay expensive handshake work until the peer proves to illustrate a concrete protocol gating pattern for handshake DoS. [14]

Client puzzles are another mitigation family which requires proof of functionality before experiencing flood testing. Handshake flooding is easiest to understand when broken into three phases. The first being initial connection establishment. The second is handshake negotiation. And the third is post handshake request processing. The experiments in this paper focus on the first two phases because that is where attackers can force repeated work without needing to understand the application.

When an endpoint fails under handshake churn, the first question should be whether the failure is caused by a cryptographic bottleneck or by a systems bottleneck. In real deployments, systems bottlenecks like connection caps or descriptor limits would trigger earlier than the CPU reaching 100%. Especially when the traffic pattern is thousands of short lived connections. [7] Protocol design can reduce some of this risk. QUIC’s anti amplification rule exists specifically because spoofed source handshakes can otherwise convert small client inputs into larger server outputs. While this paper tests TLS over TCP, the QUIC rule is still a useful reference because it formalizes a security property that TLS deployments must approximate through configuration and mitigation when they face similar adversarial patterns. [13]

DTLS 1.3’s cookie exchange demonstrates another concrete handshake gating pattern where a server can delay expensive operations until the peer proves a return route. This reduces wasted work under spoofed or non returning traffic.

Client puzzles provide a separate but complementary approach. They do not reduce

handshake bytes, but they can reduce server-side asymmetry by forcing initiators to spend computation before the server commits resources. [19] The modern IETF draft on TLS client puzzles provides an implementable starting point for evaluating puzzle based gating in TLS deployments under flood conditions. [20]

DTLS server ecosystems shows that configuration and implementation diversity can create DoS relevant behavior in the wild, which supports the core ideology that availability should be tested, not assumed. [21]

F. “Under Pressure”

In this paper, “under pressure” means two concrete stressors. (1) hybrid PQC increases handshake overhead, and (2) handshake floods force the service to pay that overhead repeatedly at scale. The first stressor is measured directly using OpenSSL handshake counters and timing loops to quantify deltas between X25519 and X25519MLKEM768 in the same environment.

The second stressor is evaluated by increasing connection churn against a TLS endpoint until failure behavior becomes visible in throughput collapse and connection errors. This framing intentionally avoids claims like “PQC is slower.” Instead, it asks whether overhead translates into measurable shifts in failure thresholds and mitigation requirements in the handshake path. [5] The IETF TLS client puzzles draft revisits this idea in a modern context, providing a standards oriented reference for handshake layer gating discussions. [20] Measurement work on DTLS servers shows that deployment realities affect handshake behavior and can create unexpected DoS relevant outcomes, motivating measurement rather than assumptions. [21]

III. PROBLEM & SOLUTION

The cryptographic correctness of ML-KEM and ML-DSA is handled by NIST standardization and leaves the question being “how does PQC change operational behavior and risk in a real handshake pipeline.” The specific risk evaluated is handshake layer DDoS, where an attacker forces repeated connection establishment to consume resources tied to handshake processing and state.

A. Threat Model

The attacker can generate high rate connection attempts that trigger full handshakes or repeated handshake negotiation. The attacker does not need to decrypt traffic or compromise endpoints. Availability degradation is the objective.

The relevant resource classes are (i) connection state and accept/backlog behavior, (ii) file descriptors and worker concurrency caps, (iii) CPU time for

handshake operations, and (iv) handshake byte volume which can increase bandwidth and parsing work. [5]

B. Evaluation Methods

To keep the evaluation directly tied to availability risk, this work uses three metric families. Handshake overhead, handshake time, and handshake flood success/failure behavior.

All raw outputs were saved as plain-text artifacts and bundled off the test environment so that every reported number can be traced back to an exact command output.

For handshake bytes, the reported values are taken directly from OpenSSL’s “SSL handshake has read ... written ...” counters, which avoids subjective interpretation.

For handshake time, the reported mean is computed from the timing sample file. The sample count N is reported as the line count of that file. For handshake flood tests, the reported throughput and latency values are taken from h2load summaries and are paired with configuration snapshots of relevant limits (ulimit, Nginx worker connections) to support causal interpretation of failure modes.

Handshake overhead is captured via OpenSSL-reported handshake bytes read and written, which provides a direct measure of how much extra data the handshake exchanges under different negotiated groups.

Handshake time is measured through repeated handshake loops with $N = 160$ samples per configuration, producing a mean time per handshake in the testbed.

Handshake flood behavior is measured under high concurrency using a TLS endpoint under test and a load generator that creates connection churn; outputs include success/failure counts and connect/TTFB distributions.

C. Configurations Under Comparison

The baseline configuration is a classical TLS 1.3 key agreement using X25519. The hybrid PQ configuration uses the group X25519MLKEM768, which aligns with ML-KEM-768-level security targets and hybrid migration guidance. [4]

D. Success Criteria

Success for this project is not defined as ‘PQC is fast.’ Instead, success is defined as producing reproducible measurements that quantify how hybrid PQ key agreement changes handshake overhead and demonstrating, through handshake flood testing, what system limits and mitigations most strongly influence failure thresholds in handshake heavy conditions.

IV. EXPERIMENT

A. Testbed

All experiments were conducted in an Ubuntu environment using OpenSSL 3.0.2 with oqsprovider loaded through the OpenSSL 3 provider interface. [6], [11] PQ and hybrid algorithm availability was confirmed by verifying provider activation and enumerating available KEM and signature algorithms through OpenSSL. [6]

B. TLS Configurations Under Test

We evaluated two TLS 1.3 key agreement configurations:

1. **Classical baseline:** X25519.
2. **Hybrid PQ group:** X25519MLKEM768.

Hybrid selection aligns with the IETF hybrid approach and uses a post-quantum component corresponding to ML-KEM-768-level security. [4][2]

C. Metrics Collected

Handshake overhead metrics (OpenSSL):

- Handshake bytes read/written (SSL handshake has read ... written ...).
- Handshake time per connection (recorded via /usr/bin/time), using repeated handshake loops with sample size $N = 160$.

Handshake flood metrics (TLS endpoint stress):

- Success/failure counts under high concurrency.
- Connect time and time to first byte distributions.
- Observed failure modes (client connect failure vs server error).

D. Mitigation

Mitigation was evaluated at the configuration level by adjusting practical limits tied to handshake layer availability, including connection capacity and file descriptor limits, because these are common failure points under high connection churn even when crypto itself is not the only bottleneck. [5] Where protocol-level mitigations exist (e.g., DTLS cookies), they are referenced as design analogies but not claimed as directly implemented in TCP based TLS in this testbed.

V. RESULTS

Based on the measured failure behavior, the safest way to treat PQC deployment is to assume the handshake is the attack surface and harden it before rolling out PQ groups broadly. At a minimum, operators should validate (1) file descriptor ceilings are above expected peak connection churn, (2) server connection concurrency settings match reality, and (3) the deployment has an explicit handshake flood mitigation plan (rate limiting, gating, or puzzle based approaches where appropriate). [7] Client puzzles are a relevant reference because they directly target the asymmetry problem. Additionally, during a flood, the server should not do expensive work for free.

The modern IETF client puzzles draft shows that this is still an active standards area topic, which supports treating handshake layer gating as a legitimate mitigation category rather than an idea. Separately, DTLS 1.3's cookie exchange is a concrete example of protocol level gating that delays expensive work until return routing is proven, which reinforces that handshake DoS resistance is a first class design concern.

This paper does not claim that one mitigation solves all cases. The point is that PQC increases handshake overhead, so the environment must be configured so that the handshake remains sustainable under churn and does not fail early due to avoidable resource caps.

A. Handshake Byte Overhead

We measured handshake byte volume for X25519 versus X25519MLKEM768 using OpenSSL's handshake byte counters.

- X25519: read 1347 B, written 287 B
- X25519MLKEM768: read 2435 B, written 1463 B

This aligns with prior findings that PQ/hybrid key agreement increases handshake message size and can increase server response volume relative to client requests. [5]

Table I summarizes the measured handshake bytes for X25519 versus X25519MLKEM768 in the OpenSSL testbed.

Group	Handshake bytes read (B)	Handshake bytes written (B)
X25519	1347	287
X25519MLKEM768	2435	1463

Table I. TLS 1.3 Handshake byte overhead

B. Handshake Time Overhead

We recorded handshake timing samples for both configurations with $N = 160$. The mean handshake time increased when using X25519MLKEM768 compared to X25519 in this testbed. This is consistent with the expectation that hybrid key agreement adds additional operations and processing steps relative to pure ECDHE. [4]

Table II reports mean handshake time for each configuration with $N = 160$ samples. Times are reported in milliseconds (ms).

Group	N	Mean handshake time (ms)
X25519	160	0.1875
X25519MLKEM768	160	1.4375

Table II. TLS 1.3 handshake timing comparison ($N=160$)

C. Handshake Flood Threshold

Figure 1 (h2load summary) captures the baseline behavior of a TLS endpoint under increasing concurrency and shows that connect and time to first byte means rise substantially as concurrency increases, even before full failure occurs.

```
bryce@capstone-pqc-tls:~/pqc-ddos-tls$ ./pqc-ddos-tls/make_summary.sh
Wrote: /home/bryce/pqc-ddos-tls/results/processed/summary.csv
Wrote: /home/bryce/pqc-ddos-tls/results/processed/summary.txt
Preview:
node | concurrency | req_per_s | connect_mean_ms | ttfb_mean_ms | failed | timeouts | elapsed_ms
-----|-----|-----|-----|-----|-----|-----|-----
baseline | 50 | 39474.20 | 3.65 | 7.19 | 0 | 0 | 253.33
baseline | 200 | 25885.54 | 151.29 | 223.43 | 0 | 0 | 386.32
baseline | 500 | 13604.17 | 786.57 | 502.97 | 0 | 0 | 735.07
```

Fig. 1. Baseline TLS 1.3 handshake flood summary (Nginx + h2load).

```
127.0.0.1:80: [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80] [127.0.0.1:80:80]
bryce@capstone-pqc-tls:~/pqc-ddos-tls$ ulimit -n
sudo cat /proc/sys/fs/file-max
1024
9223372036854775807
bryce@capstone-pqc-tls:~/pqc-ddos-tls$ sudo sysctl net.core.somaxconn
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 512
bryce@capstone-pqc-tls:~/pqc-ddos-tls$ ss -s
ss -tan | awk '{print $1}' | sort | uniq -c
Total: 270
TCP: 7 (estab 1, closed 0, orphaned 0, timewait 0)

Transport Total IP IPv6
RAW 1 0 1
UDP 2 2 0
TCP 7 5 2
INET 10 7 3
FRAG 0 0 0

1 ESTAB
6 LISTEN
1 State
bryce@capstone-pqc-tls:~/pqc-ddos-tls$ grep -R "worker_connections|worker_rlimit_nofile" /etc/nginx/nginx.conf /etc/nginx/conf.d/* 2>/dev/null
/etc/nginx/nginx.conf: worker_connections 768;
bryce@capstone-pqc-tls:~/pqc-ddos-tls$
```

Fig. 2. Observed host and server limits relevant to handshake flood behavior (ulimit, backlog, worker_connections).

Figure 2 documents the concrete system caps that shaped the failure threshold under connection churn. This figure summarizes how throughput and connect/TTFB latency change as handshake concurrency increases. Under high concurrency, a distinct threshold emerges where a large fraction of connection attempts fail, and the dominant observed

failures are connection-establishment errors rather than application layer exceptions.

Fig. 2 also shows that practical system constraints such as a low per process and file descriptor limit can determine the failure threshold under handshake churn, which reinforces that PQC-ready TLS must include systems hardening and stress testing. [7]

D. Interpreting the Measured Overhead in an Availability Context Time Overhead

The measured handshake byte and time deltas are not presented as universal constants. They are presented as concrete evidence that hybrid key agreement changes the handshake cost profile in a real stack. In this environment, the hybrid handshake increased bytes written from 287 B to 1463 B and increased bytes read from 1347 B to 2435 B, which increases bandwidth and parsing work per connection attempt. Similarly, the mean handshake time increased from 0.1875 ms (X25519) to 1.4375 ms (X25519MLKEM768) with $N = 160$, which is an order of magnitude change in handshake cost at the connection establishment boundary.

From the handshake flood perspective, even small per handshake increases matter because the attacker is attempting to multiply cost by forcing repeated handshakes. That is the core reason PQ-TLS overhead must be evaluated under flood conditions rather than only under normal traffic. [5]

The flood experiments reinforce that the failure threshold is frequently determined by a combination of overhead and system caps. When the endpoint hits a resource ceiling, the observed symptom fails and degraded time to first byte, not a cryptographic exception. For example, file descriptors or worker connection limits

VI. DISCUSSION

A. Why Increased Handshake Size Matters Under Flooding

Handshake byte growth matters because it increases bandwidth and processing overhead at the exact phase an attacker targets for availability degradation. [5] Sosnowski et al. explicitly describe PQ TLS asymmetry and note that large server responses compared to small client requests can be exploitable, and they compare this to QUIC's strict anti-amplification behavior. [12] While TLS over TCP does not enforce QUIC's anti amplification rule, the design principle remains relevant, which is that larger handshake responses increase the cost of servicing handshake attempts under flood conditions.

B. Authentication (ML-DSA) as an Availability Multiplier

This paper’s measured hybrid evaluation focused on key agreement overhead, but PQ authentication is expected to further increase handshake overhead through larger certificate chains and transcript signatures. [10] Sikeridis et al. show that PQ certificate sizes and CertificateVerify payloads can materially affect handshake time and server throughput, which implies that ML-DSA deployment should be evaluated under handshake flood conditions, not only under normal load. [16] Mixed certificate chain approaches may reduce migration risk but also introduce additional complexity that should be measured for availability impact.

C. Mitigation Tradeoffs and “PQC-Ready” Deployment Criteria

A critical outcome of this work is that PQ migration should include availability hardening and stress testing as a formal requirement. Hybrid key exchange is a reasonable transition approach, but it increases measured overhead and should be treated as a change in handshake cost profile that must be validated under worst case load. [5] Client puzzles and gating have been studied for TLS to reduce asymmetry under attack conditions, and they provide a concrete direction for mitigating handshake floods beyond basic rate limiting. [19] DTLS 1.3’s cookie exchange demonstrates that protocol-level gating is a recognized mitigation approach in handshake protocols, even if TCP-based TLS deployments rely on different mechanisms in practice.

VII. RELATED WORK

Sosnowski et al. measure post-quantum TLS 1.3 performance using blackbox and whitebox methodology and identify both data volume effects and computational cost distributions across participating libraries. [5] Their work explicitly flags attack relevant properties: server-side CPU cost can exceed client-side costs by multiple factors, and server handshake responses can be much larger than client requests depending on the chosen signature algorithm. [5]

Sikeridis et al. evaluate post-quantum authentication in TLS 1.3 and show that certificate chain size and signature operations can materially affect handshake time and server throughput under realistic conditions, which is directly relevant to availability when handshakes are forced repeatedly. [10] Kampanakis and Childs-Klein evaluate the impact of data heavy post-quantum TLS on time to last byte for real-world connections, reinforcing that “TLS overhead” is not just handshake time but also interacts with network variability and data volume. [15]

Transition strategies for post-quantum authentication include mixed certificate chains designed to improve deployability and reduce the risk of hard ecosystem breaks during migration. [16] On constrained systems, PQ TLS has been evaluated with embedded stacks (e.g., mbed TLS) and hybrid PQ TLS variants, which matters because lower compute budgets can raise susceptibility to handshake stress. [17], [18]

Handshake layer mitigations have been studied in the TLS context through client puzzles and gating approaches that reduce server asymmetry under flood conditions. [20] Modern protocol design also reflects this concern: QUIC limits server response amplification before address validation, showing that handshake layer abuse is a first class design consideration at transport level. [12]

VIII. Limitations and Future Work

This work measures hybrid key agreement overhead and handshake flood behavior in a controlled testbed and does not yet include full ML-DSA certificate chain deployment in the measured handshake overhead results. [3] PQ authentication overhead has been shown to materially affect handshake time and throughput, so integrating ML-DSA certificates and repeating the same measurements is necessary for a full deployment evaluation. [10] Future work also includes repeating measurements on constrained environments and embedded stacks, where reduced CPU budgets and memory may shift handshake flood thresholds significantly. [17] Hybrid post-quantum TLS has been evaluated on embedded devices and shows that overhead depends strongly on the device and network environment, reinforcing the need for context specific measurement. [18] Time to last byte analysis further suggests that overhead interpretation must consider data volume and network variability rather than only handshake time in isolation. [15]

IX. Conclusion

NIST standardization makes post-quantum migration in TLS a near term engineering requirement rather than a future only discussion. In this testbed, enabling the hybrid group X25519MLKEM768 increased measured handshake bytes and increased mean handshake time relative to X25519. This showed handshake flood testing showed that practical system limits can determine failure thresholds under connection churn. The main takeaway is that PQC-ready TLS must include handshake layer stress testing and configuration hardening so that cryptographic improvements do not introduce avoidable availability regressions during handshake heavy attack conditions. [7]

REFERENCES

- [1] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," 2018. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [2] National Institute of Standards and Technology (NIST), "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," 2024. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>
- [3] National Institute of Standards and Technology (NIST), "FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)," 2024. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>
- [4] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid key exchange in TLS 1.3," 2025. Available: <https://www.ietf.org/archive/id/draft-ietf-tls-hybrid-design-16.txt>
- [5] M. Sosnowski et al., "The Performance of Post-Quantum TLS 1.3," in *Proc. ACM CoNEXT Companion '23*, 2023, doi: 10.1145/3624354.3630585. Available: <https://dl.acm.org/doi/10.1145/3624354.3630585>
- [6] Open Quantum Safe, "oqsprovider: OpenSSL 3 provider for post-quantum cryptography," GitHub repository, 2024. Available: <https://github.com/open-quantum-safe/oqs-provider/blob/main/USAGE.md>
- [7] S. A. Crosby and D. S. Wallach, "Denial of Service via Algorithmic Complexity Attacks," in *Proc. 12th USENIX Security Symp.*, 2003. Available: https://www.usenix.org/legacy/events/sec03/tech/full_papers/crosby/crosby.pdf
- [8] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Foundations of Computer Science (FOCS)*, 1994, doi: 10.1109/SFCS.1994.365700.
- [9] M. Mosca "Cybersecurity in an era with quantum computers: will we be ready?," 2015. Available: <https://eprint.iacr.org/2015/1075.pdf>
- [10] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-Quantum Authentication in TLS 1.3: A Performance Study," in *Proc. NDSS Symp.*, 2020, doi: 10.14722/ndss.2020.24203. Available: <https://www.ndss-symposium.org/wp-content/uploads/2020/02/24203-paper.pdf>
- [11] R. Rios, J. A. Montenegro, A. Munoz, and D. Ferraris, "Towards the Quantum-Safe Web: Benchmarking Post-Quantum TLS," 2025. <https://ieeexplore.ieee.org/document/10844321>
- [12] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," 2021. <https://www.rfc-editor.org/rfc/rfc9000.pdf>
- [13] J. Iyengar and M. Thomson, "Using TLS to Secure QUIC," 2021. Available: <https://www.rfc-editor.org/rfc/rfc9001>
- [14] E. Rescorla, "The Datagram Transport Layer Security (DTLS) Protocol Version 1.3," 2022. Available: <https://www.rfc-editor.org/rfc/rfc9147>
- [15] P. Kampanakis and W. Childs-Klein, "The impact of data-heavy, post-quantum TLS 1.3 on the Time-To-Last-Byte of real-world connections," 2024. Available: <https://eprint.iacr.org/2024/176>
- [16] S. Paul, Y. Kuzovkova, N. Lahr, and R. Niederhagen, "Mixed Certificate Chains for the Transition to Post-Quantum Authentication in TLS 1.3," in *Proc. ACM ASIA CCS*, 2022, doi: 10.1145/3488932.3497755. Available: <https://dl.acm.org/doi/10.1145/3488932.3497755>
- [17] K. Burstinghaus-Steinbach, C. Kraub, R. Niederhagen, and M. Schneider, "Post-Quantum TLS on Embedded Systems: Integrating and Evaluating Kyber and SPHINCS+ with mbed TLS," in *Proc. ACM ASIA CCS*, 2020, doi: 10.1145/3320269.3384725. Available: <https://dl.acm.org/doi/10.1145/3320269.3384725>
- [18] D. Marchsreiter and J. Sepúlveda, "Hybrid Post-Quantum Enhanced TLS 1.3 on Embedded Devices," in *Proc. Euromicro Conf. on Digital System Design (DSD)*, 2022, doi: 10.1109/DSD57027.2022.00127.
- [19] D. Dean and A. Stubblefield, "Using Client Puzzles to Protect TLS," in *Proc. 10th USENIX Security Symp.*, 2001. Available: <https://www.usenix.org/conference/10th-usenix-security-symposium/using-client-puzzles-protect-tls>
- [20] E. Venhoek, "TLS Client Puzzles," 2024. Available: <https://datatracker.ietf.org/doc/draft-venhoek-tls-client-puzzles/>
- [21] N. Erinola et al., "Analysis of the DTLS Server Ecosystem on the Internet," in *Proc. USENIX Security Symp.*, 2023. Available: <https://www.usenix.org/system/files/usenixsecurity23-erinola.pdf>